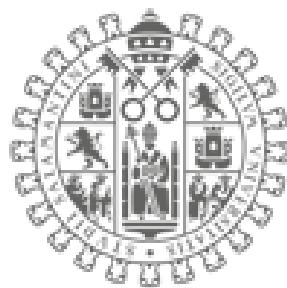


Diseño Administración e Integración de Infraestructuras de Tecnologías de la In- formación

Terraform



**VNiVERSiDAD
D SALAMANCA**

CAMPUS OF INTERNATIONAL EXCELLENCE

Miguel Barranquero Diez

Índice

1. Introducción	3
1.1. Objetivo del Proyecto	3
1.2. Descripción General	3
1.3. Configuración de Credenciales	3
2. Configuración del Entorno	4
2.1. Requisitos Previos	4
2.2. Instalación de AWS CLI	4
2.3. Configuración de AWS CLI	4
2.4. Generación de Claves SSH	4
2.5. Obtención de AMIs	4
3. Estructura del Proyecto	6
3.1. Árbol de Archivos	6
3.2. Descripción de Archivos	6
3.2.1. main.tf	6
3.2.2. variables.tf	6
3.2.3. outputs.tf	6
4. Código Terraform	7
4.1. Parte 1: Configuración del Proveedor AWS	7
4.2. Parte 2: Variables	7
4.3. Parte 3: Claves SSH	8
4.4. Parte 4: Security Groups	9
4.4.1. Security Group 1 (SSH + HTTP)	9
4.4.2. Security Group 2 (SSH + HTTPS)	9
4.4.3. Security Group 3 (SSH + MySQL)	10
4.5. Parte 5: Instancias EC2	12
4.5.1. Instancia 1: EC2-1 (Amazon Linux)	12
4.5.2. Instancia 2: EC2-2 (Ubuntu)	12
4.5.3. Instancia 3: EC2-3 (Red Hat)	12
4.6. Parte 6: Outputs	13
5. Proceso de Despliegue	14
5.1. Comandos Ejecutados	14
5.2. Resultado de terraform apply	15
6. Evidencias del Despliegue	17
6.1. Dashboard de AWS EC2	17
6.2. Detalles de las Instancias	17
6.3. Security Groups	18
6.4. Outputs de Terraform	19
7. Verificación del Funcionamiento	20
7.1. Tabla Resumen de la Infraestructura	20
7.2. Conexión SSH a las Instancias	20
7.3. Verificación de Puertos Abiertos	20
8. Limpieza de Recursos	21
9. Conclusión	22
Bibliografía	23

1. Introducción

1.1. Objetivo del Proyecto

El presente proyecto tiene como objetivo desplegar una infraestructura en Amazon Web Services (AWS) mediante el uso de Infrastructure as Code (IaC), específicamente utilizando Terraform. La infraestructura consistirá en tres máquinas virtuales (instancias EC2) independientes, cada una con su propia configuración de seguridad y autenticación.

1.2. Descripción General

Se desplegarán tres instancias EC2 de forma simultánea en la región de Estocolmo (`eu-north-1`), cada una con:

- **Sistema operativo diferente:** Amazon Linux 2023, Ubuntu 22.04 LTS y Red Hat Enterprise Linux 9
- **Tipo de instancia específico:** t3.small, c7i-flex.large y m7i-flex.large respectivamente
- **Clave SSH única:** key1, key2 y key3
- **Security Group personalizado:** con reglas de firewall específicas para cada máquina

1.3. Configuración de Credenciales

Nota importante sobre seguridad: En este proyecto se utiliza AWS CLI para la gestión de credenciales, **NO se incluyen credenciales hardcodedas** en los archivos de Terraform. Las credenciales se configuran mediante:

```
aws configure
```

Este comando almacena las credenciales de forma segura en `~/.aws/credentials`, evitando exponer información sensible en el código fuente. Esta es una práctica recomendada de seguridad [1].

2. Configuración del Entorno

2.1. Requisitos Previos

Antes de iniciar el despliegue, se instalaron las siguientes herramientas en el sistema Ubuntu.

2.2. Instalación de AWS CLI

La instalación de AWS CLI se realizó mediante los siguientes comandos:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o  
"awscliv2.zip"  
unzip awscliv2.zip  
sudo ./aws/install
```

2.3. Configuración de AWS CLI

Se configuraron las credenciales de AWS especificando la región de trabajo:

```
aws configure  
# AWS Access Key ID: [REDACTED]  
# AWS Secret Access Key: [REDACTED]  
# Default region name: eu-north-1  
# Default output format: json
```

2.4. Generación de Claves SSH

Se generaron tres pares de claves SSH RSA de 4096 bits para cada instancia:

```
ssh-keygen -t rsa -b 4096 -f key1 -N "" -C "key1-terraform"  
ssh-keygen -t rsa -b 4096 -f key2 -N "" -C "key2-terraform"  
ssh-keygen -t rsa -b 4096 -f key3 -N "" -C "key3-terraform"
```

Cada comando genera dos archivos:

- **Clave privada** (key1 , key2 , key3): Se mantienen seguras y NO se comparten
- **Clave pública** (key1.pub , key2.pub , key3.pub): Se suben a AWS

2.5. Obtención de AMIs

Para garantizar la compatibilidad con la región `eu-north-1` , se consultaron las AMIs disponibles mediante AWS CLI:

```
# Amazon Linux 2023  
aws ec2 describe-images --region eu-north-1 --owners amazon \  
--filters "Name=name,Values=al2023-ami-2023.*-x86_64" \  
--query 'Images | sort_by(@, &CreationDate) | [-1].[ImageId]' \  
--output text  
  
# Ubuntu 22.04 LTS  
aws ec2 describe-images --region eu-north-1 --owners 099720109477 \  
--filters "Name=name,Values=ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-  
server-*" \  
--query 'Images | sort_by(@, &CreationDate) | [-1].[ImageId]' \  
--output text
```

```
# Red Hat Enterprise Linux 9
aws ec2 describe-images --region eu-north-1 --owners 309956199498 \
  --filters "Name=name,Values=RHEL-9.*-x86_64-*" \
  --query 'Images | sort_by(@, &CreationDate) | [-1].[ImageId]' \
  --output text
```

3. Estructura del Proyecto

3.1. Árbol de Archivos

La estructura del proyecto Terraform es la siguiente:

```
terraform-DAIITI/  
├─ main.tf           # Configuración principal de recursos  
├─ variables.tf      # Definición de variables  
├─ outputs.tf        # Valores de salida  
├─ key1              # Clave privada SSH (NO compartir)  
├─ key1.pub          # Clave pública SSH  
├─ key2              # Clave privada SSH (NO compartir)  
├─ key2.pub          # Clave pública SSH  
├─ key3              # Clave privada SSH (NO compartir)  
└─ key3.pub          # Clave pública SSH
```

3.2. Descripción de Archivos

3.2.1. main.tf

Contiene la configuración principal de todos los recursos a desplegar:

- Configuración del proveedor AWS
- Definición de claves SSH (aws_key_pair)
- Security Groups con reglas de firewall
- Instancias EC2

3.2.2. variables.tf

Define las variables utilizadas en el proyecto, incluyendo:

- Región de AWS
- IDs de las AMIs para cada sistema operativo
- CIDR blocks para acceso SSH

3.2.3. outputs.tf

Especifica los valores que se mostrarán al finalizar el despliegue:

- IPs públicas de cada instancia
- Detalles completos (tipo, clave SSH, security group)

4. Código Terraform

4.1. Parte 1: Configuración del Proveedor AWS

El archivo `main.tf` comienza con la configuración del proveedor AWS [2]:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.region
}
```

Explicación:

- `required_providers`: Define que necesitamos el proveedor de AWS
- `version = "~> 5.0"`: Acepta versiones 5.x (permite actualizaciones menores)
- `region = var.region`: Utiliza la variable definida en `variables.tf` (eu-north-1)

4.2. Parte 2: Variables

El archivo `variables.tf` contiene las definiciones de variables necesarias:

```
variable "region" {
  description = "Region de AWS donde desplegar los recursos"
  type        = string
  default     = "eu-north-1"
}

variable "ami_amazon_linux" {
  description = "AMI de Amazon Linux 2023 en eu-north-1"
  type        = string
  default     = "ami-0705384c0b33c194c"
}

variable "ami_ubuntu" {
  description = "AMI de Ubuntu 22.04 LTS en eu-north-1"
  type        = string
  default     = "ami-0014ce3e52359afbd"
}

variable "ami_redhat" {
  description = "AMI de Red Hat Enterprise Linux en eu-north-1"
  type        = string
  default     = "ami-0f265f8a93c5e8f8e"
}
```

4.3. Parte 3: Claves SSH

Se crean tres recursos `aws_key_pair` para registrar las claves públicas en AWS:

```
resource "aws_key_pair" "key1" {
  key_name     = "key1"
  public_key   = file("${path.module}/key1.pub")

  tags = {
    Name = "key1"
  }
}

resource "aws_key_pair" "key2" {
  key_name     = "key2"
  public_key   = file("${path.module}/key2.pub")

  tags = {
    Name = "key2"
  }
}

resource "aws_key_pair" "key3" {
  key_name     = "key3"
  public_key   = file("${path.module}/key3.pub")

  tags = {
    Name = "key3"
  }
}
```

Explicación:

- `file()` : Lee el contenido del archivo de clave pública
- `path.module` : Variable que apunta al directorio actual del módulo

4.4. Parte 4: Security Groups

Se definen tres Security Groups con reglas específicas para cada instancia [3].

4.4.1. Security Group 1 (SSH + HTTP)

```
resource "aws_security_group" "securitygroup1" {
  name           = "securitygroup1"
  description    = "Permite SSH y HTTP"

  ingress {
    description = "SSH from anywhere"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    description = "All outbound traffic"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "securitygroup1"
  }
}
```

4.4.2. Security Group 2 (SSH + HTTPS)

```
resource "aws_security_group" "securitygroup2" {
  name           = "securitygroup2"
  description    = "Permite SSH y HTTPS"

  ingress {
    description = "SSH from anywhere"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
ingress {
  description = "HTTPS from anywhere"
  from_port   = 443
  to_port     = 443
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

egress {
  description = "All outbound traffic"
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}

tags = {
  Name = "securitygroup2"
}
```

4.4.3. Security Group 3 (SSH + MySQL)

```
resource "aws_security_group" "securitygroup3" {
  name           = "securitygroup3"
  description    = "Permite SSH y MySQL"

  ingress {
    description = "SSH from anywhere"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "MySQL from anywhere"
    from_port   = 3306
    to_port     = 3306
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    description = "All outbound traffic"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
```

```
    Name = "securitygroup3"  
  }  
}
```

Explicación de reglas:

- `ingress` : Reglas de tráfico entrante (inbound)
- `egress` : Reglas de tráfico saliente (outbound)
- `protocol = "-1"` : Permite todos los protocolos
- `cidr_blocks = ["0.0.0.0/0"]` : Permite acceso desde cualquier IP

4.5. Parte 5: Instancias EC2

Se crean tres instancias EC2 con diferentes configuraciones [4].

4.5.1. Instancia 1: EC2-1 (Amazon Linux)

```
resource "aws_instance" "vm1" {
  ami                = var.ami_amazon_linux
  instance_type      = "t3.small"
  key_name           = aws_key_pair.key1.key_name
  vpc_security_group_ids = [aws_security_group.securitygroup1.id]

  tags = {
    Name = "EC2-1"
  }
}
```

4.5.2. Instancia 2: EC2-2 (Ubuntu)

```
resource "aws_instance" "vm2" {
  ami                = var.ami_ubuntu
  instance_type      = "c7i-flex.large"
  key_name           = aws_key_pair.key2.key_name
  vpc_security_group_ids = [aws_security_group.securitygroup2.id]

  tags = {
    Name = "EC2-2"
  }
}
```

4.5.3. Instancia 3: EC2-3 (Red Hat)

```
resource "aws_instance" "vm3" {
  ami                = var.ami_redhat
  instance_type      = "m7i-flex.large"
  key_name           = aws_key_pair.key3.key_name
  vpc_security_group_ids = [aws_security_group.securitygroup3.id]

  tags = {
    Name = "EC2-3"
  }
}
```

Parámetros explicados:

- `ami` : ID de la imagen del sistema operativo
- `instance_type` : Tipo de instancia (define CPU, RAM, almacenamiento)
- `key_name` : Nombre de la clave SSH para acceso
- `vpc_security_group_ids` : Lista de Security Groups asociados
- `tags` : Etiquetas para identificar recursos en AWS

4.6. Parte 6: Outputs

El archivo `outputs.tf` define los valores que se mostrarán tras el despliegue:

```
output "vm1_public_ip" {
  description = "IP publica de la instancia EC2-1"
  value       = aws_instance.vm1.public_ip
}

output "vm2_public_ip" {
  description = "IP publica de la instancia EC2-2"
  value       = aws_instance.vm2.public_ip
}

output "vm3_public_ip" {
  description = "IP publica de la instancia EC2-3"
  value       = aws_instance.vm3.public_ip
}

output "vm1_details" {
  description = "Detalles completos de EC2-1"
  value = {
    instance_name = "EC2-1"
    public_ip     = aws_instance.vm1.public_ip
    instance_type = aws_instance.vm1.instance_type
    key_name      = aws_instance.vm1.key_name
    security_group = aws_security_group.securitygroup1.name
  }
}

output "vm2_details" {
  description = "Detalles completos de EC2-2"
  value = {
    instance_name = "EC2-2"
    public_ip     = aws_instance.vm2.public_ip
    instance_type = aws_instance.vm2.instance_type
    key_name      = aws_instance.vm2.key_name
    security_group = aws_security_group.securitygroup2.name
  }
}

output "vm3_details" {
  description = "Detalles completos de EC2-3"
  value = {
    instance_name = "EC2-3"
    public_ip     = aws_instance.vm3.public_ip
    instance_type = aws_instance.vm3.instance_type
    key_name      = aws_instance.vm3.key_name
    security_group = aws_security_group.securitygroup3.name
  }
}
```

Los outputs permiten acceder fácilmente a información importante como las IPs públicas y confirmar la configuración de cada instancia.

5. Proceso de Despliegue

5.1. Comandos Ejecutados

El despliegue se realizó siguiendo el flujo estándar de Terraform [5]:

1. Inicialización

```
terraform init
```

Este comando:

- Descarga el proveedor de AWS (v5.100.0)
- Crea el archivo `.terraform.lock.hcl` para bloquear versiones
- Prepara el directorio de trabajo

2. Formateo del Código

```
terraform fmt
```

Formatea automáticamente el código según las convenciones de Terraform.

3. Validación

```
terraform validate
```

Verifica que la sintaxis y las referencias entre recursos sean correctas.

4. Planificación

```
terraform plan
```

Muestra una vista previa de los recursos que se crearán:

- 3 instancias EC2
- 3 Security Groups
- 3 Key Pairs

5. Aplicación

```
terraform apply
```

Ejecuta el despliegue. Terraform solicita confirmación escribiendo `yes`.

5.2. Resultado de terraform apply

La ejecución de `terraform apply` completó exitosamente el despliegue de todos los recursos:

```
aws_key_pair.key2: Refreshing state... [id=key2]
aws_key_pair.key3: Refreshing state... [id=key3]
aws_key_pair.key1: Refreshing state... [id=key1]
aws_security_group.securitygroup3: Refreshing state... [id=sg-01382741ab96880a8]
aws_security_group.securitygroup1: Refreshing state... [id=sg-05bef18ff4e9d1954]
aws_security_group.securitygroup2: Refreshing state... [id=sg-0af50b441b5553e02]
aws_instance.vm1: Refreshing state... [id=i-027eceb79e16ealc6]
aws_instance.vm3: Refreshing state... [id=i-09e9415262833342]
aws_instance.vm2: Refreshing state... [id=i-0d937835bc80e2aaf]
```

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
all_instances_summary = {
  "vm1" = {
    "ip" = "13.53.60.230"
    "key" = "key1"
    "name" = "EC2-1"
    "security_group" = "securitygroup1"
  }
  "vm2" = {
    "ip" = "16.170.226.103"
    "key" = "key2"
    "name" = "EC2-2"
    "security_group" = "securitygroup2"
  }
  "vm3" = {
    "ip" = "13.60.211.129"
    "key" = "key3"
    "name" = "EC2-3"
    "security_group" = "securitygroup3"
  }
}
vm1_details = {
  "instance_name" = "EC2-1"
  "instance_type" = "t3.small"
  "key_name" = "key1"
  "public_ip" = "13.53.60.230"
  "security_group" = "securitygroup1"
}
vm1_public_ip = "13.53.60.230"
vm2_details = {
  "instance_name" = "EC2-2"
  "instance_type" = "c7i-flex.large"
```

```
    "key_name" = "key2"
    "public_ip" = "16.170.226.103"
    "security_group" = "securitygroup2"
  }
  vm2_public_ip = "16.170.226.103"
  vm3_details = {
    "instance_name" = "EC2-3"
    "instance_type" = "m7i-flex.large"
    "key_name" = "key3"
    "public_ip" = "13.60.211.129"
    "security_group" = "securitygroup3"
  }
  vm3_public_ip = "13.60.211.129"
```

El despliegue creó exitosamente:

- 3 instancias EC2 (vm1, vm2, vm3)
- 3 Security Groups (securitygroup1, securitygroup2, securitygroup3)
- 3 Key Pairs (key1, key2, key3)

Tiempo total de ejecución: Aproximadamente 2-3 minutos.

6. Evidencias del Despliegue

6.1. Dashboard de AWS EC2

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...
EC2-1	i-027ecb79e16ea1c6	running	t3.small	3/3 checks passed	View alarms +	eu-north-1c	ec2-13-53-60-230.eu-n...	13.53.60.230
Ejemplo2	i-019385705a9d17b95	Shutting-down	t3.micro	-	View alarms +	eu-north-1b	ec2-13-61-15-250.eu-n...	13.61.15.250
EC2-2	i-0d937835bc80e2aaf	running	c7i-flex.large	3/3 checks passed	View alarms +	eu-north-1b	ec2-16-170-226-103.eu...	16.170.226.103
EC2-3	i-09e94152628333342	running	m7i-flex.large	3/3 checks passed	View alarms +	eu-north-1b	ec2-13-60-211-129.eu...	13.60.211.129

Figure 1: Vista del dashboard de EC2 mostrando las tres instancias en estado “running”

En la imagen se observan las tres instancias desplegadas:

- **EC2-1:** Amazon Linux 2023, tipo t3.small
- **EC2-2:** Ubuntu 22.04 LTS, tipo c7i-flex.large
- **EC2-3:** Red Hat Enterprise Linux 9, tipo m7i-flex.large

Todas las instancias se encuentran en estado “running” y ubicadas en la región **eu-north-1** (Estocolmo).

6.2. Detalles de las Instancias

EC2-1: Amazon Linux

Configuración:

- **AMI:** Amazon Linux 2023 (ami-0705384c0b33c194c)
- **Instance Type:** t3.small
- **Key Pair:** key1
- **Security Group:** securitygroup1
- **Puertos abiertos:** SSH (22), HTTP (80)

EC2-2: Ubuntu

Configuración:

- **AMI:** Ubuntu 22.04 LTS (ami-0014ce3e52359afbd)
- **Instance Type:** c7i-flex.large
- **Key Pair:** key2
- **Security Group:** securitygroup2
- **Puertos abiertos:** SSH (22), HTTPS (443)

EC2-3: Red Hat

Configuración:

- **AMI:** Red Hat Enterprise Linux 9 (ami-0f265f8a93c5e8f8e)
- **Instance Type:** m7i-flex.large
- **Key Pair:** key3
- **Security Group:** securitygroup3
- **Puertos abiertos:** SSH (22), MySQL (3306)

6.3. Security Groups

☐	Name	Security group ID	Security group name	VPC ID	Description	Owner
☐	-	sg-0b802a686c1ed5dd5	default	vpc-07eca5ea82077e85a	default VPC security group	554099414786
☐	securitygroup3	sg-01382741ab96880a8	securitygroup3	vpc-07eca5ea82077e85a	Permite SSH y MySQL	554099414786
☐	securitygroup2	sg-0af50b441b5553e02	securitygroup2	vpc-07eca5ea82077e85a	Permite SSH y HTTPS	554099414786
☐	securitygroup1	sg-05bef18ff4e9d1954	securitygroup1	vpc-07eca5ea82077e85a	Permite SSH y HTTP	554099414786

Figure 2: Vista general de los tres Security Groups creados

Se observan los tres Security Groups con sus respectivos IDs y configuraciones de VPC.

Reglas de Security Group 1

Reglas de entrada:

- Puerto 22 (SSH): Protocolo TCP, origen 0.0.0.0/0
- Puerto 80 (HTTP): Protocolo TCP, origen 0.0.0.0/0

Reglas de salida:

- Todo el tráfico: Protocolo All, destino 0.0.0.0/0

Reglas de Security Group 2

Reglas de entrada:

- Puerto 22 (SSH): Protocolo TCP, origen 0.0.0.0/0
- Puerto 443 (HTTPS): Protocolo TCP, origen 0.0.0.0/0

Reglas de salida:

- Todo el tráfico: Protocolo All, destino 0.0.0.0/0

Reglas de Security Group 3

Reglas de entrada:

- Puerto 22 (SSH): Protocolo TCP, origen 0.0.0.0/0
- Puerto 3306 (MySQL): Protocolo TCP, origen 0.0.0.0/0

Reglas de salida:

- Todo el tráfico: Protocolo All, destino 0.0.0.0/0

6.4. Outputs de Terraform

Los valores de salida generados por Terraform tras el despliegue exitoso:

Outputs:

```
vm1_public_ip = "13.53.60.230"
vm2_public_ip = "16.170.226.103"
vm3_public_ip = "13.60.211.129"

vm1_details = {
  instance_name = "EC2-1"
  public_ip     = "13.53.60.230"
  instance_type = "t3.small"
  key_name      = "key1"
  security_group = "securitygroup1"
}

vm2_details = {
  instance_name = "EC2-2"
  public_ip     = "16.170.226.103"
  instance_type = "c7i-flex.large"
  key_name      = "key2"
  security_group = "securitygroup2"
}

vm3_details = {
  instance_name = "EC2-3"
  public_ip     = "13.60.211.129"
  instance_type = "m7i-flex.large"
  key_name      = "key3"
  security_group = "securitygroup3"
}
```

Estos outputs permiten:

- Acceder rápidamente a las IPs públicas para conexión SSH
- Verificar que cada instancia tiene la configuración correcta
- Confirmar las asociaciones entre instancias, claves y security groups

7. Verificación del Funcionamiento

7.1. Tabla Resumen de la Infraestructura

Instancia	SO	Tipo	IP Pública	Key	Puertos
EC2-1	Amazon Linux	t3.small	13.53.60.230	key1	22, 80
EC2-2	Ubuntu 22.04	c7i-flex.large	16.170.226.103	key2	22, 443
EC2-3	RHEL 9	m7i-flex.large	13.60.211.129	key3	22, 3306

7.2. Conexión SSH a las Instancias

Para verificar el acceso a las instancias, se pueden utilizar los siguientes comandos:

```
# Ajustar permisos de las claves privadas
chmod 400 key1 key2 key3

# Conexión a EC2-1 (Amazon Linux)
ssh -i key1 ec2-user@13.53.60.230

# Conexión a EC2-2 (Ubuntu)
ssh -i key2 ubuntu@16.170.226.103

# Conexión a EC2-3 (Red Hat)
ssh -i key3 ec2-user@13.60.211.129
```

Nota: Los usuarios por defecto varían según el sistema operativo:

- Amazon Linux y Red Hat: `ec2-user`
- Ubuntu: `ubuntu`

7.3. Verificación de Puertos Abiertos

Se puede verificar que los puertos están correctamente configurados:

```
# Verificar puerto HTTP en EC2-1
curl -I http://13.53.60.230

# Verificar puerto HTTPS en EC2-2
curl -I https://16.170.226.103

# Verificar puerto MySQL en EC2-3
nc -zv 13.60.211.129 3306
```

8. Limpieza de Recursos

Para evitar costes innecesarios, es fundamental destruir los recursos cuando finalice el proyecto:

```
# Ver qué recursos se destruirán
terraform plan -destroy

# Destruir todos los recursos
terraform destroy
```

Terraform solicitará confirmación escribiendo `yes` . El proceso eliminará en orden:

1. Las instancias EC2
2. Los Security Groups
3. Los Key Pairs

Tiempo estimado de destrucción: 1-2 minutos.

9. Conclusión

Este proyecto ha demostrado la eficacia de Terraform como herramienta de Infrastructure as Code para el despliegue automatizado de recursos en AWS. La capacidad de definir infraestructura compleja mediante código declarativo simplifica enormemente la gestión de recursos en la nube, permitiendo despliegues consistentes, reproducibles y escalables [6].

El uso de AWS CLI para la gestión de credenciales, junto con la correcta configuración de Security Groups y claves SSH, establece una base sólida de seguridad para la infraestructura desplegada.

Bibliografía

- [1] A. W. Services, “AWS Command Line Interface Documentation.” [Online]. Available: <https://docs.aws.amazon.com/cli/>
- [2] HashiCorp, “AWS Provider - Terraform Registry.” [Online]. Available: <https://registry.terraform.io/providers/hashicorp/aws>
- [3] A. W. Services, “Security Groups for Your VPC.” [Online]. Available: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html
- [4] A. W. Services, “Amazon EC2 User Guide for Linux Instances.” [Online]. Available: <https://docs.aws.amazon.com/ec2/>
- [5] HashiCorp, “Terraform Documentation.” [Online]. Available: <https://www.terraform.io/docs>
- [6] K. Morris, *Infrastructure as Code: Managing Servers in the Cloud*, 2nd ed. O'Reilly Media, 2020.